

Understanding Your LoRa P2P Firmware

A Parameter-by-Parameter Teaching Guide
Wio-E5 mini · STM32WLE5 · SubGHz_Phy P2P Ping-Pong Project

Reference project: `Wio-e5-mini-p2p-tx / Wio-e5-mini-p2p-rx`
Config file: `SubGHz_Phy/App/subghz_phy_app.h`

Prepared as a teaching reference — every setting below is explained using the actual code from this project, so it can be read directly alongside the firmware.

Contents

1. Your Current Configuration (the code we are explaining)
2. Radio Frequency & Power
3. Bandwidth & Spreading Factor
4. Coding Rate — explained in depth with a worked example
5. Preamble, Symbol Timeout & Packet Format Settings
6. RSSI vs SNR — "how loud" vs "how clear"
7. The RTC (Real Time Clock) — what it does in this project today
8. Interrupts and CPU Time
9. Message Framing & Addressing
10. Summary Table: AC-powered vs Battery-powered Ideal Values

1. Your Current Configuration

Everything in this document explains the block below, taken directly from your transmitter's configuration file. This is the block we will go through, line by line.

Wio-e5-mini-p2p-tx / SubGHz_Phy / App / subghz_phy_app.h

```
#define RF_FREQUENCY                865100000 /* Hz */
#define TX_OUTPUT_POWER            14          /* dBm, conducted (RFO_HP path) */

#define LORA_BANDWIDTH             0           /* 0: 125 kHz */
#define LORA_SPREADING_FACTOR     7           /* SF7 */
#define LORA_CODINGRATE           1           /* 1: 4/5 */
#define LORA_PREAMBLE_LENGTH      8           /* symbols */
#define LORA_SYMBOL_TIMEOUT       5           /* symbols */
#define LORA_FIX_LENGTH_PAYLOAD_ON false
#define LORA_IO_INVERSION_ON      false
#define LORA_CRC_ENABLED          true

#define BUFFER_SIZE                64
#define RX_TIMEOUT_VALUE           3000      /* ms */
```

This same block (with matching values) also exists in the receiver project, `Wio-e5-mini-p2p-rx`, because **both sides of a P2P link must agree on almost every one of these settings** to understand each other — like two people needing to agree on the same language before they can talk.

2. Radio Frequency & Power

RF_FREQUENCY — which channel you're using

Current value: 865100000 Hz = 865.1 MHz

Think of this like tuning a radio dial to a specific FM station. Both the transmitter and receiver must be tuned to the exact same frequency, or they simply won't hear each other — like two people trying to talk on different phone lines.

What happens if you change it: A different frequency doesn't make the signal stronger or weaker by itself — it just needs to (a) match on both TX and RX, and (b) stay inside the legal license-free band for your country. 865-867 MHz is the de-licensed band in India, so 865.1 MHz is a safe, legal choice. Changing this without a good reason isn't necessary.

TX_OUTPUT_POWER — how loud you shout

Current value: 14 dBm **Hardware capability:** up to 22 dBm (confirmed in `radio_board_if.c`, function `RBI_GetRFOMaxPowerConfig`)

dBm is just a unit for radio "loudness." This is the one setting that is purely about the transmitter — it is how strong the signal is *at the moment it leaves your antenna*, before it starts losing strength on its journey to the receiver.

Analogy: This is how loud you shout at your own mouth. It is not how loud it sounds to the person listening far away — that's a different measurement (RSSI, covered in Section 6) taken at the other end, after the shout has traveled and weakened.

What happens if you increase it (toward 22): Longer range, because the signal starts out stronger and can travel further before becoming too weak to hear. On AC power this costs nothing. On battery power, this draws more current for the brief moment each message is sent.

What happens if you decrease it: Shorter range, but less current drawn per transmission — useful only if you are trying to save every last bit of battery and don't need the extra distance.

3. Bandwidth & Spreading Factor

LORA_BANDWIDTH — how wide your "road" is

Current value: 0 = 125 kHz

Analogy: Imagine your radio channel as a road. A wider road (higher bandwidth, e.g. 250 kHz or 500 kHz) lets more data through per second — faster, but each "car" (bit of data) is harder to see clearly from far away. A narrower road (125 kHz, your current setting) is slower, but each car is easier to spot from a distance.

What happens if you increase it: Faster data transfer, but shorter range — not useful for your range goal.
What happens if you decrease it (e.g. 62.5 kHz): Range improves slightly further, but the radio becomes far more sensitive to small clock/crystal errors, and messages take noticeably longer to send. **125 kHz is the recommended setting to keep — it is the practical sweet spot.**

LORA_SPREADING_FACTOR — how slowly you spell things out

Current value: 7 (SF7) **Range:** can be set from SF7 (fastest, shortest range) to SF12 (slowest, longest range)

Analogy: Talking across a valley. SF7 is talking at a normal pace — understood nearby, lost far away. SF12 is saying every syllable slowly and deliberately — takes much longer to say the same sentence, but carries much further.

What happens if you increase it (toward SF12): Significantly longer range. But: each message takes roughly **10-15x longer** to transmit at SF12 than at SF7, so the radio (and the CPU driving it) stays "busy" much longer per message. On battery power, this is the single biggest range-vs-battery trade-off in the whole configuration.
What happens if you decrease it: Faster messages, shorter range. SF7 (current) is already the fastest, shortest-range end of the scale.

Important side-effect: If you raise the Spreading Factor, you must also raise `RX_TIMEOUT_VALUE` and any retry/ping-interval delays in `subghz_phy_app.c` to match — otherwise the receiver will give up waiting before the (now much longer) message has even finished arriving, and you'll see false "timeout" / "disconnected" messages.

4. Coding Rate — In Depth

LORA_CODINGRATE — how much "repair material" you add

Current value: `1 = 4/5` (from the code comment `/* 1: 4/5 */`)

LoRa breaks your data into small groups of **4 bits** of real information. Before sending each group over the air, it adds a few extra bits — not new data, just *repair/backup* bits — so that if a few bits get flipped by radio noise during the journey, the receiver can detect and even fix the mistake without asking for a resend.

The setting is written as `4/x`: the first number (always 4) is the real data bits in one group; the second number is the *total* bits actually sent for that group (real + repair).

Code value	Ratio	Real bits	Repair bits added	Total bits sent	Meaning
1 (current)	4/5	4	1	5	Least protection, least extra airtime
2	4/6	4	2	6	A bit more protection
3	4/7	4	3	7	More protection
4	4/8	4	4	8	Maximum protection — literally double the airtime

Worked example using your own ping message

Your transmitter sends the text `"PING:5"` (see `SendPing()` in `subghz_phy_app.c`) — 6 characters = 48 bits of real data.

48 bits × (5 ÷ 4) ≈ 60 bits actually transmitted — at your current 4/5 setting

48 bits × (8 ÷ 4) = 96 bits actually transmitted — at the strongest 4/8 setting

Analogy: This is bubble wrap on a package. 4/5 is a thin layer — fine for gentle handling (a clean radio environment), ships quickly. 4/8 is wrapping the package four times over — survives much rougher handling (a noisy radio environment with interference), but takes noticeably more time and space to ship. The contents of the box (your actual data) never change — only how well-protected it is in transit.

What happens if you increase it (toward 4/8): Much better chance the message survives a noisy or marginal-signal environment without being thrown away as corrupted. Costs more airtime per message (up to double).

What happens if you decrease it: Faster messages, but more likely to be rejected as corrupted in weak-signal conditions — more retries needed, which can end up costing more time/battery overall than the "slower but more reliable" option.

5. Preamble, Timeout & Packet Format Settings

LORA_PREAMBLE_LENGTH — the "warm-up buzz"

Current value: 8 symbols

Analogy: Clearing your throat before speaking, so the listener knows to start paying attention. This is a short, repeating signal sent before the real message, letting the receiver's radio lock onto the transmission.

A longer preamble slightly increases airtime on every message but improves how reliably the receiver can lock on, especially at high Spreading Factors. 8 symbols is a sensible default and does not need changing for this project.

LORA_SYMBOL_TIMEOUT — the receiver's patience while listening for that buzz

Current value: 5 symbols

This is how many preamble symbols the receiver is willing to wait through before giving up on hearing anything at all. Too short, and it might give up right before a real message arrives; too long, and it keeps its radio switched on (burning power) even with nothing coming.

LORA_FIX_LENGTH_PAYLOAD_ON — fixed-size vs variable-size messages

Current value: `false` (variable length)

This decides whether every message must be exactly the same size, or can vary. Your current ping/ack messages are short text strings of different lengths ("PING:5", "ACK:5", "PONG"), so variable-length is currently the right choice. If you move to the fixed-size address+data frame described in Section 9, this should be switched to `true` — fixed-length headers are slightly shorter and marginally more robust over the air.

LORA_IQ_INVERSION_ON — a handshake agreement, not a range setting

Current value: `false` on both TX and RX

This is a signal-processing convention that must simply *match* on both ends — it doesn't affect range, speed, or power at all. Think of it as agreeing which side is "up" before shaking hands; get it wrong and the two sides can't understand each other at all, but getting it "right" doesn't make anything faster or further-reaching.

LORA_CRC_ENABLED — the built-in "is this message damaged?" check

Current value: `true`

A small checksum added to every message. The receiver uses it to instantly detect if a message got corrupted in transit, and silently throws it away rather than handing your application garbled data. Adds only 2 bytes per message — a negligible cost for the safety it provides.

Recommended: always keep this on.

BUFFER_SIZE — not a radio setting at all

Current value: 64 bytes

This is simply how much memory (RAM) is set aside on the microcontroller to hold one message. It has no effect on the radio, range, speed, or power — it just needs to be big enough to hold your longest message. 64 bytes is comfortably enough even for the addressed binary frame proposed later in this document.

RX_TIMEOUT_VALUE — how long the receiver keeps listening for a reply

Current value: 3000 ms

After sending a ping, the transmitter switches its radio into "listen" mode and waits up to this long for a reply before giving up. This value must be long enough to cover the time it actually takes for the far side to receive, process, and reply — which grows a lot if you raise the Spreading Factor (Section 3).

Battery note: "Radio actively listening" is one of the most power-hungry states there is. If a reply never comes, the radio burns through the *entire*

timeout window every single time doing nothing useful. On a battery-powered design, this value should be calculated from your actual chosen Spreading Factor's timing, with only a small safety margin — not padded generously "just in case."

6. RSSI vs SNR — "How Loud" vs "How Clear"

RSSI (Received Signal Strength Indicator)

This is measured at the **receiver**, and tells you how strong the message was by the time it arrived — after traveling through the air, walls, weather, and distance.

$$\text{RSSI} = \text{TX_OUTPUT_POWER} - \text{everything lost along the way}$$

Analogy: You shout at 14 dBm (your mouth). By the time the sound crosses a field and reaches your friend, it might only register as −100 dBm (his ear) — it dropped a lot along the way ("path loss"). Both numbers describe "how strong," just measured at two different ends of the journey.

Key teaching point: You only ever get an RSSI number for a message that was *already received successfully*. There is no such thing as "the RSSI of a message that never arrived." So RSSI alone should never be used as your "no signal" rule — a message either arrives (with some RSSI number, however weak) or it doesn't arrive at all, which shows up as a **timeout**, not a bad RSSI reading. Your code already handles this correctly: see `OnRxTimeout()` and `OnRxError()` in `subghz_phy_app.c`, which flip the link state to "disconnected."

SNR (Signal to Noise Ratio)

This tells you how much clearer your message was compared to the radio background "hiss" (noise) at the moment of arrival. Unusually for radio systems, LoRa can successfully decode messages even when this number is *negative* — the message is technically "quieter than the noise" and it still works, thanks to how LoRa spreads its signal.

Practical use: Watch SNR trending downward over time as your early warning that you're approaching the edge of range — it is a more meaningful "getting weak" indicator than RSSI, especially near the limit.

7. The RTC (Real Time Clock)

The RTC is a small clock/calendar chip on the board. In this project it is switched on at startup (`MX_RTC_Init()` in `main.c`), but it is **not actually used for anything** in the application logic.

Core/Src/timer_if.c — the bridge that is supposed to connect the RTC to the application

```
UTIL_TIMER_Status_t TIMER_IF_StartTimer(uint32_t timeout)
{
    UTIL_TIMER_Status_t ret = UTIL_TIMER_OK;
    /* USER CODE BEGIN TIMER_IF_StartTimer */

    /* USER CODE END TIMER_IF_StartTimer */
    return ret;
}
```

Every function in this file is an empty shell like the one above — the RTC ticks, but nothing reads it. Instead, your actual message pacing (waiting 1.5 seconds between pings, etc.) is done with a much simpler blocking delay in `SubghzApp_Process()`, unrelated to the RTC.

Does this matter today? No — it's simply unused, not broken. It only becomes important if you later want the microcontroller to *sleep* between pings to save battery, and use the RTC to wake itself back up on schedule. That would require actually filling in `timer_if.c` and arming an RTC wake-up interrupt — a good next project for students once the basic link is understood.

Extra technical detail: The RTC clock source is configured as `RCC_RTCCLKSOURCE_LSI` (see `stm32wlxx_hal_msp.c`) — an internal, low-accuracy oscillator, similar to a cheap watch that slowly gains or loses time. If the RTC is used for real scheduling later, switching to the crystal-based LSE source (if populated on the board) would give far more accurate timekeeping.

8. Interrupts and CPU Time

An interrupt is a signal the radio chip sends to the processor meaning "stop what you're doing and handle me right now" — like a phone ringing during a task.

```
Middleware/.../stm32_radio_driver/radio.c
```

```
static void RadioOnDioIrq( RadioIrqMasks_t radioIrq )
{
    SubgRf.RadioIrq = radioIrq;
    RADIO_IRQ_PROCESS(); /* handled immediately, inside the interrupt itself */
}
```

In this project, every radio event (message sent, message received, timeout, error) is handled **immediately and completely** inside the interrupt, at the highest priority the chip supports (`HAL_NVIC_SetPriority(SUBGHZ_Radio_IRQn, 0, 0)` in `stm32wlxx_hal_msp.c`). This is the normal, intended design for this chip, and the actual work done — copying about 60 bytes and flagging a task — takes a tiny fraction of a millisecond.

The real slowdown is elsewhere. Look at `SubghzApp_Process()` in `subghz_phy_app.c`:

```
Wio-e5-mini-p2p-tx / SubGHz_Phy / App / subghz_phy_app.c
```

```
HAL_Delay(PING_TX_PERIOD_MS); /* 1500 ms — freezes EVERYTHING, every single round-trip */
SendPing();
```

This deliberate pause freezes the entire application — not just radio handling, but any other task the processor might otherwise be doing — for 1.5 seconds every cycle. That is roughly **a thousand times longer** than the radio interrupt itself takes to run. If more "free CPU time" for other application work is the goal, replacing this blocking delay with a non-blocking, scheduled approach (ideally tied to the RTC from Section 7) is by far the highest-impact change — not tuning the interrupt.

9. Message Framing & Addressing

What your code sends today

```
subghz_phy_app.c — SendPing() (TX) and the reply logic (RX)
```

```
snprintf((char *)Buffer, BUFFER_SIZE, "PING:%lu", (unsigned long)PktSeqNum);
/* RX replies with: "ACK:<seq>", "PONG", or "ACK" — all plain text, no address field */
```

This works today only because there is exactly one transmitter and one receiver — nobody else is listening, so there's no confusion about who a message is meant for. It will not scale once more device pairs share the same channel: any device would try to respond to any message it overhears.

A proposed fixed binary frame (3-byte address + 3-byte data, as requested)

Byte offset	Field	Size	Purpose
0	Message Type	1 byte	e.g. 0x01 = PING, 0x02 = ACK/PONG, 0x03 = DATA
1-3	Address	3 bytes	Identifies which device pair this message belongs to
4-6	Real-time Data	3 bytes	The actual sensor/application data being carried
7	Sequence Number	1 byte	Replaces the current text-based counter; wraps 0-255

Total: 8 fixed bytes — comfortably inside `BUFFER_SIZE` (64). Once every message is this fixed shape, set `LORA_FIX_LENGTH_PAYLOAD_ON` to `true` (Section 5) for a small extra saving. Because `LORA_CRC_ENABLED` is already `true`, a corrupted frame is discarded by the radio hardware itself before your code ever sees it — so an additional application-level checksum is not necessary here.

10. Summary Table — Ideal Values

Parameter	Current	Ideal (AC power, max range)	Ideal (battery power, balanced)
TX_OUTPUT_POWER	14 dBm	22 dBm (hardware max)	17–20 dBm
LORA_BANDWIDTH	125 kHz	125 kHz (keep)	125 kHz (keep)
LORA_SPREADING_FACTOR	SF7	SF12 (max range)	SF9–SF10 (balanced)
LORA_CODINGRATE	4/5	4/8 (max robustness)	4/5 or 4/6
LORA_PREAMBLE_LENGTH	8 symbols	8–12 symbols	8 symbols (keep)
RX_TIMEOUT_VALUE	3000 ms	Sized to match chosen SF's airtime	As tight as possible for chosen SF (avoid padding)
RTC / Sleep	Unused, always awake	Not required	Highest priority fix: sleep between pings, wake via RTC

End of document. All code excerpts above are taken directly from the `wio-e5-mini-p2p-tx` and `wio-e5-mini-p2p-rx` STM32CubeIDE projects (SubGHz_Phy P2P ping-pong example, STM32WLE5 / Wio-E5 mini).